

eTRON Developer Manual

Supervised by: Shakif Aziz

Sponsored by: Micromax

Developed by InSync:

Noah Bradley

Rhys Cleary

Matthew Page

Matthew Parkinson

Holly Wyatt

Table of Contents

Architecture Summary	2
Frontend	2
Backend	2
Lambda Functions	2
Authentication and Storage	2
Setup Guide	3
Required Applications	3
Pre-Setup	3
Android Studio	3
EAS Account	3
Setup	4
Frontend Development	4
Backend Console Configurations and Development	4
Amplify	4
AWS Console	5
DynamoDB	5
S3 Event Notifications (Requires pre-deployment)	6
EventBridge (Requires pre-deployment)	7
Simple Queue Service (SQS)	9
AWS AppSync	10
Deployment	12
Device Deployment	12
AWS Deployment	12

Architecture Summary

Frontend

The frontend files with eTRON's visible pages are inside *etron/frontend/app*.

Backend

Lambda Functions

The lambda functions are inside *etron/backend* and are found based on their dataset (domain). Each folder contains a folder for the lambda functions and a *template.yaml* file that defines the API patterns.

- *etron/backend/workspace*
- *etron/backend/modules/day-book/[feature]*
- *etron/backend/user*
- *etron/backend/audit*

Each folder contains a shared-folder with common functions to each feature.

Authentication and Storage

These are created with Amplify and are defined at *etron/frontend/amplify-backend/amplify/[type]*.

Inside the *auth/resource.ts* file the type of login is defined as email. It also enables login with external providers, Google and Microsoft. This file defines the custom attributes for a user, *has_workspace* and *country_code*. *Has_workspace* takes a boolean value as a string and is used to verify if a user is part of a workspace. The *country_code* is associated with phone numbers.

The file *storage/resource.ts* defines the three S3 buckets used within eTRON. Each define their own access permissions, currently only authenticated users can read, write and delete objects from the users and workspace storages. *appConfigurations* is read only.

Setup Guide

This setup guide will assume identical tools used to the ones in development. Alternative IDEs, version-tracking, and emulation software are possible.

Required Applications

- VSCode
- GitHub Desktop
- Android Studio

Pre-Setup

Android Studio

1. Open Android Studio.
2. Select "More Actions" -> "Virtual Device Manager".
3. Select the Plus icon "Create Virtual Device".
4. Select device (we recommend Pixel 6 Pro).
5. Select "Next".
6. Select the most recent System Image
7. Continue through the steps until "Finish" is available.
8. Go to "More Actions" -> "SDK Manager".
9. Copy the SDK location. It should appear as something similar to "C:\Users\<Name>\AppData\Local\Android\Sdk".
10. In Windows search, search for "edit environmental variables for your account".
11. Click "New".
12. For Variable Name, enter: ANDROID_HOME.
13. For Variable Value, paste the SDK location.
14. Press "OK" to save it.
15. The Android Emulator should be ready to use now.

EAS Account

For initial setup of the development environment and to build/deploy the app to mobile devices or the app/play store you will need to set up an account with Expo Application Services (EAS).

<https://expo.dev/eas>

Setup

Frontend Development

1. Navigate to etron/frontend in Command Prompt.
 2. Run: `npm install`
 3. Run: `npx expo install expo-dev-client`
 4. Run: `npm install -g eas-cli && eas login`
 5. Login with Expo username and password.
 6. Run: `eas build --platform android --profile development`
 7. Say “Yes” to installing the emulator.
 8. Select the emulator set up in Android Studio.
-
1. While the emulator is running, in etron/frontend, run: `npx expo run:android`
 2. Select your emulator, if necessary.
 3. After installing, it should open a QR code that can be used for running on a physical device, but that’s not necessary for emulation.
 4. Make sure it is “using development build”; press s to swap to dev build if not.
 5. Press a to open with Android.
 6. The emulator will now open the app and update as the frontend files are edited (and saved).

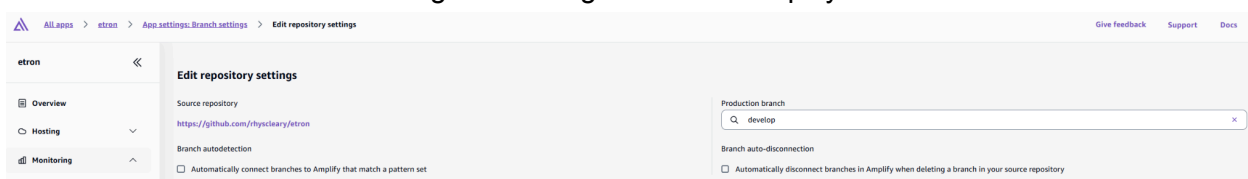
Backend Console Configurations and Development

The backend is split between SAM for defining Lambdas and APIs; and AWS Amplify Gen2 for authentication and bucket configurations. Only some services will require manual configuration which is outlined below. It should be noted, some configurations such as S3 notifications and EventBridge rules can only be set up once the infrastructure is initially deployed from SAM.

Amplify

Under etron/frontend you will see a folder amplify-backend. This contains configurations for authentication and storage (S3).

In the AWS Console you will have to create a new app and connect it to your repository. We set eTRON’s amplify configurations to automatically deploy to AWS based on a branch in our version control. This can be configured through the AWS Amplify service.



Follow the Amplify Docs to assist in setup:

<https://docs.amplify.aws/react/start/quickstart/>

AWS Console

Configuration Files

After deploying Amplify, navigate to S3 and copy the folders modules/ and permissions/ that are located within etron/app-configurations to the appconfigurationbucket.

DynamoDB

1. Create Tables

When creating these tables all you need to configure is the table name, partition key and sort key. Leave the other configurations as default.

Table Name	Partition Key	Sort Key
AuditLogs	workspaceId	logId
DataSources	workspaceId	dataSourceId
Metrics	workspaceId	metricId
Reports	workspaceId	sk
WorkspaceInvites	workspaceId	inviteId
Workspaces	workspaceId	sk
WorkspaceUsers	workspaceId	userId

2. Set up a Time To Live (TTL) for an invite in WorkspaceInvites based on the expireAt attribute.

3. You now need to set up Global secondary indexes (GSIs)

All you need to do is define the Partition key, Sort key (if it has one) and Index name for each index.

Create global secondary index [Info](#)

Global secondary indexes allow you to perform queries on attributes that are not part of a table's primary key.

Index details [Info](#)

Partition key

Enter the partition key name

1 to 255 characters.

Sort key - optional

Enter the sort key name

1 to 255 characters.

Index name

Type the index name

Between 3 and 255 characters. Only A-Z, a-z, 0-9, underscore characters, hyphens, and periods allowed.

Leave the remaining configurations as default.

Table	Partition Key	Sort Key	Index name
AuditLogs	userId	workspaceId	UserLogsIndex
WorkspaceInvites	email	-	email-index
	workspaceId	email	workspaceId-email-index
Workspaces	ownerId	-	ownerId-index
	workspaceId	moduleKey	workspaceId-moduleKey-index
WorkspaceUsers	userId	-	userId-index
	workspaceId	email	workspaceId-email-index

S3 Event Notifications (Requires pre-deployment)

These are events that are triggered when an object is placed into an S3 bucket.

- Create a fileUploadEvent on the Workspaces bucket (This event handles local data sources like the upload of csv files):
 - Select the bucket with a name like:
amplify-d3vtak5uri6g9a-de-workspacesbucket784af028-utfi3tbysihb
 - Select properties and scroll down to event notifications. Select create event notification.
 - Configure a new Put event with these configurations:

Send a notification when specific events occur in your bucket. [Learn more](#)

☐	Name	Event types	Filters	Destination type	Destination
☐	DataSourceFileUpload	Put	workspaces/	Lambda function	etron-backend-DayBookFunc-DataSourceUploadProcess-jl61P17XmeI

- The destination Lambda is the DataSourceUploadProcessorFunction.
- d. Select save changes

EventBridge (Requires pre-deployment)

eTRON has two event rules set; both of these invoke Lambda functions at intervals. DataSourcePollInitialiser does not require any manual configuration and is defined within the SAM day-book template.

```
DataSourcePollInitialiser:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri: ./data-sources/functions/polling/data-source-poller-init
    Handler: initialiserHandler.handler
    Timeout: 30
    Environment:
      Variables:
        STATE_MACHINE_ARN: !GetAtt DataSourcePollingStateMachine.Arn
    Policies:
      - DynamoDBCrudPolicy:
          TableName: DataSources
      - DynamoDBCrudPolicy:
          TableName: Workspaces
      - S3WritePolicy:
          BucketName: !Ref WorkspaceBucketName
      - Statement:
          - Effect: Allow
            Action:
              - states:StartExecution
            Resource: !Ref DataSourcePollingStateMachine
          - Effect: Allow
            Action:
              - ssm:GetParameter
              - ssm:PutParameter
              - ssm:DeleteParameter
              - ssm:GetParameters
            Resource: "*"
          - Effect: Allow
            Action:
              - sqs:SendMessage
              - s3:GetObject
              - s3:PutObject
              - s3:ListBucket
              - s3:DeleteObject
              - s3:GetBucketLocation
              - s3:HeadObject
            Resource: "*"
    Events:
      PollSchedule:
        Type: Schedule
        Properties:
          Schedule: rate(15 minutes)
```

Polling of each remote data source is scheduled at 15 minute intervals.

The second event is the Audit archiver that archives audit logs from DynamoDB into long-term Amazon S3 Glacier storage after 30 days. This event runs once per day and scans the AuditLog table for entries 30 days past creation.

Follow the steps below to setup the event bridge rule:

1. Navigate to Amazon EventBridge

2. Select Rules from the side menu
3. Select create rule
4. Change the selection from build to configure
5. Use the following configurations:

Define rule detail [Info](#)

Scheduled rule detail

Name

DailyAuditArchiver

Maximum of 64 characters consisting of numbers, lower/upper case letters, -, _, .

Description - optional

Enter description

Event bus [Info](#)

Custom or partner event bus is not supported when Schedule is selected.

default

☒ Enable the rule on the selected event bus

Define schedule [Info](#)

Schedule pattern

Schedule pattern

Choose the schedule type that best meets your needs.

☒ A fine-grained schedule that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.

☐ A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression [Info](#)

Define the cron expression for the schedule

☒ cron ()

Minutes

Hours

Day of month

Month

Day of week

Year

Next 10 trigger date(s)

UTC

Sat, 15 Nov 2025 00:00:00 UTC
 Sun, 16 Nov 2025 00:00:00 UTC
 Mon, 17 Nov 2025 00:00:00 UTC
 Tue, 18 Nov 2025 00:00:00 UTC
 Wed, 19 Nov 2025 00:00:00 UTC
 Thu, 20 Nov 2025 00:00:00 UTC
 Fri, 21 Nov 2025 00:00:00 UTC
 Sat, 22 Nov 2025 00:00:00 UTC
 Sun, 23 Nov 2025 00:00:00 UTC
 Mon, 24 Nov 2025 00:00:00 UTC

Select target(s)

Permissions

Note: When using the EventBridge console, EventBridge will automatically configure the proper permissions for the selected targets. If you're using the AWS CLI, SDK, or CloudFormation, you'll need to configure the proper permissions.

Target 1

Target types

Select an EventBridge event bus, EventBridge API destination (SaaS partner), or another AWS service as a target.

- ☐ EventBridge event bus
☐ EventBridge API destination
☒ AWS service

Select a target

Select target(s) to invoke when an event matches your event pattern or when schedule is triggered (limit of 5 targets per rule)

Lambda function

Target location

- ☒ Target in this account
☐ Target in another AWS account

Function

etron-backend-AuditDomainFun-AuditArchiverFunction-MwR57fqDzoC9

Configure version/alias

Permissions

- ☐ Use execution role (recommended)

Additional settings

Additional settings

Configure target input

You can customize the text from an event before EventBridge passes the event to the target of a rule.

Matched events

Retry policy

A retry policy determines the maximum number of hours and number of times to retry sending an event to a target after an error occurs.

Maximum age of event - optional

The maximum number of hours to keep unprocessed events for. The default value is 24 hours.

24 hour(s) 00 minute(s) 00 second(s)

Retry attempts - optional

The maximum number of times to retry sending an event to a target after an error occurs. The default value is 185 times.

185 time(s)

Dead-letter queue

Unprocessed events can be sent to standard SQS queue

- ☒ None
☐ Select an Amazon SQS queue in the current AWS account to use as the dead-letter queue
☐ Specify an Amazon SQS queue in other AWS account as a dead-letter queue

6. Continue through the options keeping the default values. Then create the rule.

Simple Queue Service (SQS)

This is used for queuing the audit log events. When an audit event is triggered the message is queued there before being processed into DynamoDB.

1. Navigate to SQS service
2. Press create queue

3. Under details select Standard and give the queue the name AuditQueue
4. Under configuration change the retention period to 14 days
5. Leave the other settings untouched
6. Scroll and select Create queue

AWS AppSync

This is used for real-time refreshes of data in the frontend. Not all of this has been implemented but there are schemas defined ready to be implemented.

1. Navigate to AWS AppSync
2. Create a new API with the type GraphQL. Leave it with the default configurations.

Select API type

API type Info

☒ **GraphQL APIs**
A GraphQL API manages a single API by providing a single endpoint to query or update data securely from multiple databases, microservices, and APIs.



☐ **Merged APIs**
A Merged API manages a group of AWS AppSync GraphQL APIs owned by different teams by composing a single GraphQL schema and securely exposing data from multiple backend APIs to authorized clients in a single API endpoint.



GraphQL API Data Source

GraphQL API Data Source
Select an option below to begin configuring your GraphQL API.

☒ **Design from scratch**
Start with a model or write your own schema.

☐ **Start with a DynamoDB table**
Create a new API from an existing DynamoDB table.

☐ **Start with an Amazon Aurora cluster**
Create a new API from a database in an existing Amazon Aurora cluster using the RDS Data API.

☐ **Create a real-time API**
Create a real-time WebSocket API where clients subscribe to channels.

3. Give it the name eTRON AppSync API
4. Within specify GraphQL resources select Create GraphQL resources later.
5. Go to Review and Create then select Create API
6. Add the schema. The schema can be seen in json format in the attached AppSyncSchema.txt file
7. On the right hand side you will see Resolvers:

The screenshot displays the AWS AppSync Studio interface. On the left, a sidebar contains navigation links: APIs, eTRON AppSync API, Schema (selected), Data sources, Functions, Queries, Caching, Settings, Monitoring, Custom domain names, and Documentation. The main area is titled 'Schema' and shows a GraphQL schema definition. The schema includes types for DataSource, Metric, and Mutation. The DataSource type has fields like workspaceId, dataSourceId, associatedMetrics, config, createdAt, updatedAt, createdBy, error, expires, lastUpdate, method, metrics, name, sourceType, and status. The Metric type has fields like workspaceId, metricId, activeDataSource, config, createdAt, updatedAt, createdBy, dataSourceId, name, and thumbnailKey. The Mutation type has a field like notifyDataUpdate. On the right, the 'Resolvers' panel shows a list of resolvers for the schema. The 'Mutation' section lists three resolvers: notifyDataUpdate, notifyMetricUpdate, and notifyDataSourceUpdate. The 'Query' section lists two resolvers: getMetricsByWorkspace and getDataSourcesByWorkspace. The 'Subscription' section lists one resolver: onDataUpdate. Each resolver has an 'Attach' button.

- a. For each mutation set the resolver data source to None
 - b. For each query set the resolver data source as shown
8. Insert the resolver code for the mutation notifyDataUpdate. See attached file DataUpdateResolver.txt.
9. Select Data Sources on the side menu. Add three different data sources:
 - a. Name - DataSourceTable, Type - AMAZON_DYNAMODB, Resource - DataSourcees
 - b. Name - MetricsTable, Type - AMAZON_DYNAMODB, Resource - Metrics
 - c. Name - None, Type - NONE

Deployment

Device Deployment

EAS services were used to build the frontend into an APK file that can be installed on android devices. The steps below assume you have already logged into eas within the previous steps.

1. Run: `npx expo prebuild`
2. To build for android run: `eas build --platform android --profile preview`

If you want to build this to install onto the app store run `eas build --platform android --profile production` instead.

AWS Deployment

Important Note: There is a resource that is defined throughout the templates that you will have to adapt to your needs. InSync simply ran out of time to adapt this, but it can be changed to an environment variable to make future deployments between accounts automated.

Each function within the template.yaml files are restricted by the user pool.

```
AuditCoreFunction:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri: ../functions/audit-core
    Handler: auditHandler.handler
    Policies:
      - DynamoDBCrudPolicy:
          TableName: Workspaces
      - DynamoDBCrudPolicy:
          TableName: WorkspaceUsers
      - DynamoDBCrudPolicy:
          TableName: AuditLogs
      - Statement:
          - Effect: Allow
            Action:
              - s3:GetObject
              - s3:PutObject
              - s3:ListBucket
              - s3:DeleteObject
            Resource: "*"
      - Effect: Allow
        Action:
          - cognito-idp:ListUsers
          - cognito-idp:AdminUpdateUserAttributes
        Resource:
          - arn:aws:cognito-idp:ap-southeast-2:841196398222:userpool/ap-southeast-2_Q4EgaQS86
```

You will need to change this to your user pool resource.

eTRON uses AWS Serverless Application Model (SAM) to write the backend infrastructure. This automates the build and deployment process to AWS. The steps below will prepare your environment and deploy the infrastructure to AWS.

1. First download the AWS CLI, this is required to authenticate to AWS.

Download:

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Verify the installation:

`aws --version`

2. Download the AWS SAM CLI to build and deploy the SAM application

Download:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/install-sam-cli.html>

Verify the installation:

`sam --version`

3. To authenticate with AWS use the command `aws configure`.

AWS Access Key ID - AKIA4HWZWM2HCZ67TL5V

AWS Secret Access Key - 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

Default region - ap-southeast-2

Default output format (if it asks leave it as json)

Note: make sure you run the following command inside the repositories /backend folder

4. This step assumes you have python3 or later installed. Run `py prep.py` to delete/create the node_modules and package-lock files/folders. This will also reinstall them.

5. Build the project with `sam build`

6. Deploy the backend with:

`sam deploy --config-env dev --parameter-overrides`

`WorkspaceBucketName=amplify-d3vtak5uri6g9a-de-workspacesbucket784af028-utfi3tbysihb`

`AppConfigurationBucketName=amplify-d3vtak5uri6g9a-de-appconfigurationbucket22-7sikap3r61rh`

`w AuditQueueArn=arn:aws:sqs:ap-southeast-2:841196398222:AuditQueue`

`AuditQueueUrl=https://sqs.ap-southeast-2.amazonaws.com/841196398222/AuditQueue`

`AppSyncUrl=https://tc2mxnxwg5hpcg26pykz7d7lea.appsync-api.ap-southeast-2.amazonaws.com/graphql AppSyncApiKey=da2-ttvwyxkrsncvjp43jpkusqbdr`

This command contains several parameters for resources manually set up in AWS.

WorkspaceBucketName and AppConfigurationBucketName are the Amazon resource

names (Arns) of the buckets. AuditQueueArn and AuditQueueUrl are the resource name

and urls of the AuditQueue event setup in Amazon Simple Queue Services. AppSyncUrl and APIKey are from the AppSync API setup in the console.